Distributed gradient descent: when the model does not fit

Part 2 of 2

Carlos Cotrini

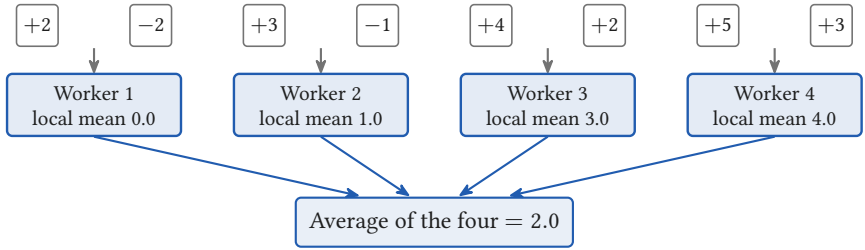
Section 1

The memory ledger

1

Four workers, one average

Eight examples, one gradient number each

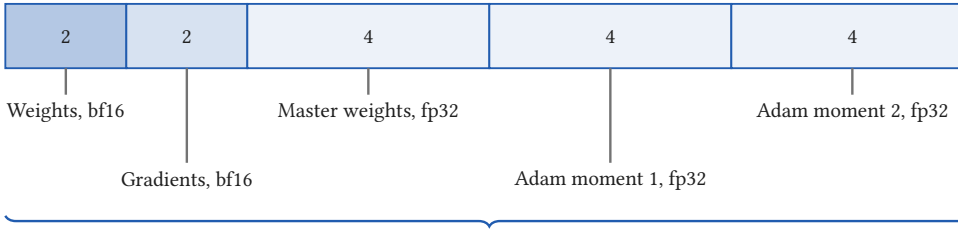


One all-reduce averages them, and it is exactly the single machine gradient

- The gradient of a mean is the mean of the gradients
- Every worker held a whole copy of the model. Today it does not fit.

Sixteen bytes per parameter

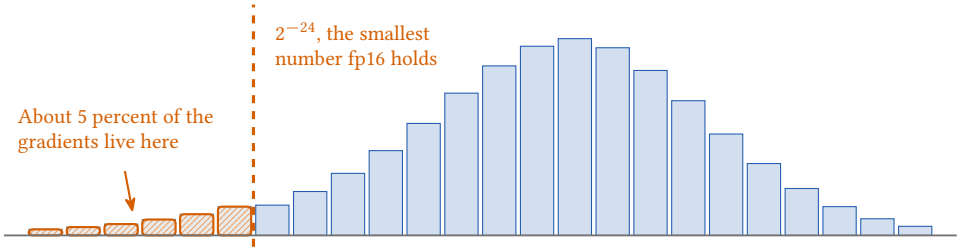
Mixed precision Adam, one parameter



$$2 + 2 + 4 + 4 + 4 = 16 \text{ bytes per parameter}$$

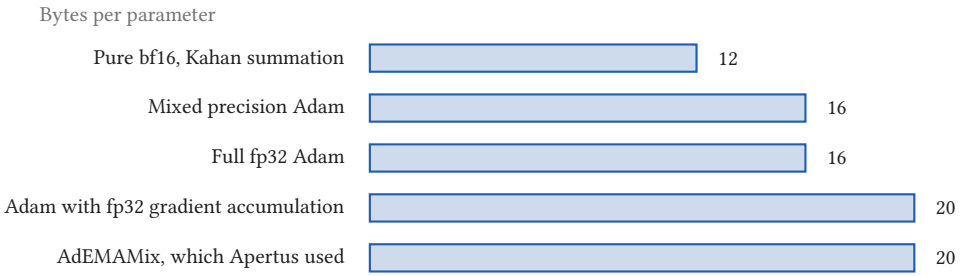
ZeRO (Rajbhandari et al., SC20) Section 3.1, verbatim: $2\Psi + 2\Psi + K\Psi = 16\Psi$ bytes, with Ψ the parameter count and $K = 12$ the optimiser multiplier, a different K from part 1's gradient buffer.

Why four of those bytes are fp32



Schematic shape. The measured fact is the 5 percent, from Micikevicius et al. 2018, Figure 2b.

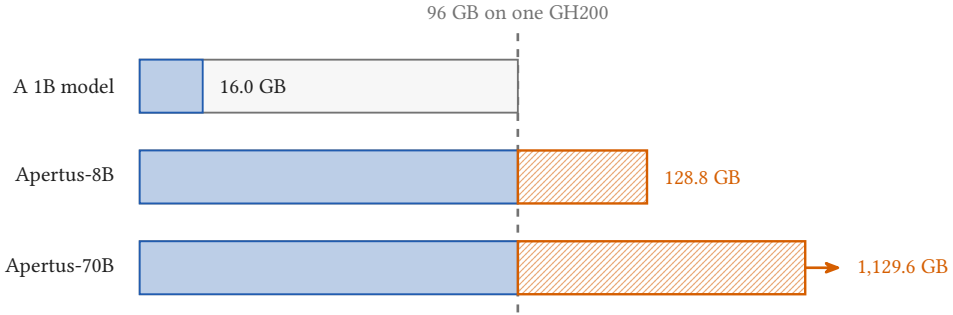
The ledger changes with the optimiser



Mixed precision does not reduce the model state. It redistributes it: full fp32 Adam costs the same 16 bytes.

ZeRO Section 3.1; Ultra-Scale Playbook; CS336 Lecture 7. AdEMAMix keeps three optimiser states, not two.

Three models, one accelerator



$$\frac{96\ \text{GB}}{16\ \text{bytes}} = 6.00 \times 10^9\ \text{parameters}$$

Apertus-8B is already past it, before a single activation

On an 80 GB card the wall is at 5.00×10^9 parameters. With Apertus's own optimiser, at 4.80×10^9.

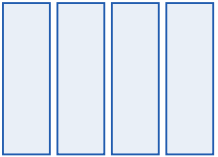
What the ledger leaves out

- **Activations**: usually the larger number at long context
- **CUDA context**: 1 to 2 GB on every GPU, before your model
- **Temporary buffers**: a flattened fp32 all-reduce buffer for a 1.5B model is 6 GB
- **Fragmentation**: ZeRO observed failures with over 30 percent nominally free

Three ways to cut it

Cut the matrix
tensor parallel

Cut the depth
pipeline parallel



Cut the state
ZeRO and FSDP

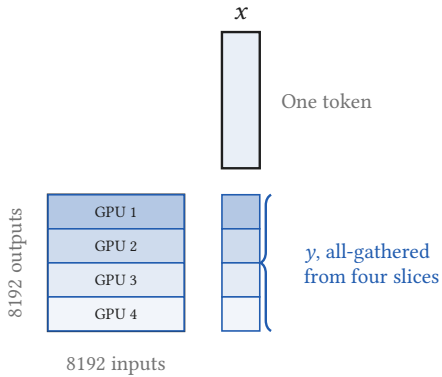
- And every large published run uses all three at once

Section 2

Cut the matrix

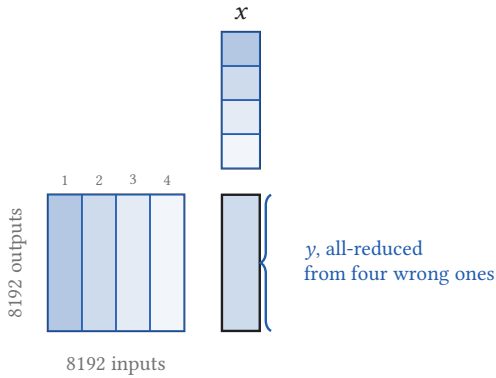
2

Cut the rows: each device owns some outputs



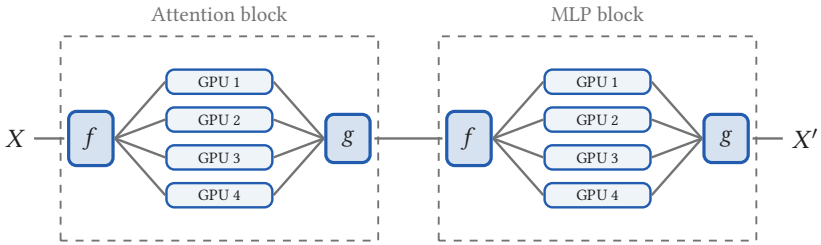
- Every device is given the whole of x
- Each owns 2048 of the 8192 outputs
- Each computes its own slice of y, alone
- The slices are concatenated, never summed

Cut the columns: each device owns some inputs



- Every device is given a slice of x
- Each owns 2048 of the 8192 inputs
- Each produces a full length answer
- Right shape, wrong value, on every device
- Only their sum is the layer's output

Four collectives per layer



f does nothing going forward, and all-reduces coming back

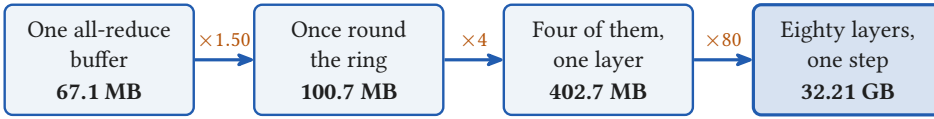
g all-reduces going forward, and does nothing coming back

Two all-reduces forward, two backward, per layer

Megatron-LM (Shoeybi et al. 2019), Section 3 and Figure 4.

Cut the matrix

One buffer, four collectives, eighty layers



$2(P-1)/P$ with $P = 4$, the tensor parallel degree

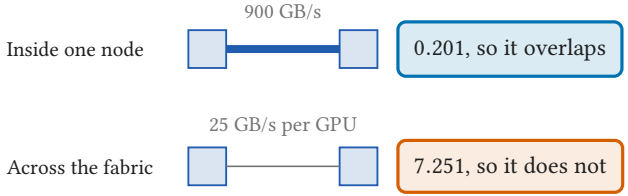
And that is per step, on top of the data parallel all-reduce from yesterday.

Apertus-70B: bf16, sequence 4096, micro batch 1, $d_{\text{model}} = 8192$, tensor parallel degree 4.

A second published expression for this volume disagrees, and disagrees in direction. Quote the collective count, not a volume.

Why tensor parallelism stays inside the node

$$\underbrace{\frac{TP-1}{2h}}_{\text{how much must move}} \times \underbrace{\frac{\text{peak FLOP/s}}{\text{peak bandwidth}}}_{\text{how fast the link is}} \leq 1$$



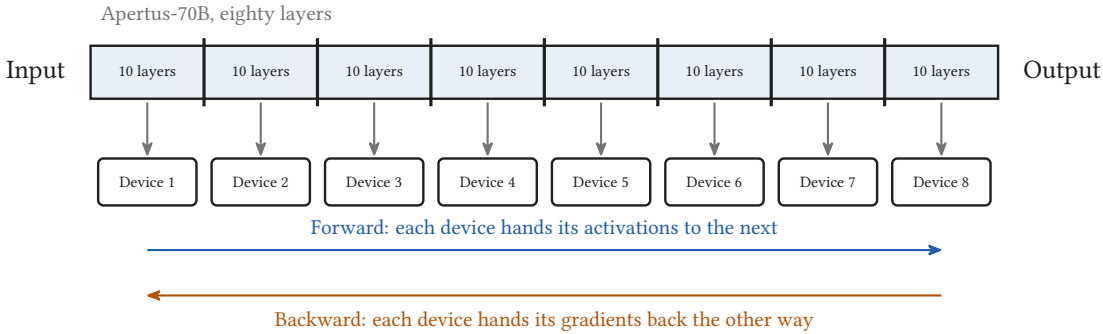
- Pinned inside the node in every large run: PTD-P at 8, Llama 3 405B at 8, Apertus-70B at 4
- Capped by the key and value heads as well, and Apertus has 8

Section 3

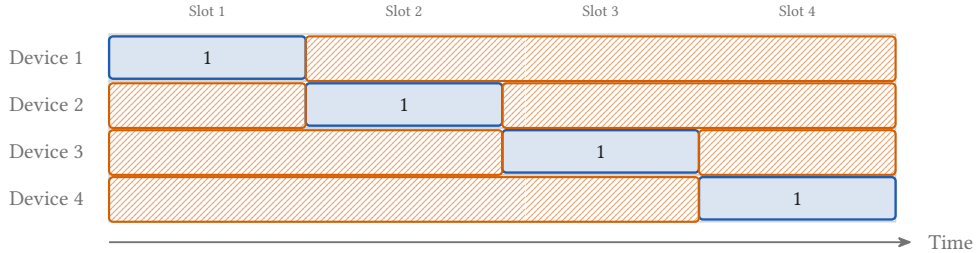
Cut the depth, and the bubble

3

Eighty layers over eight devices

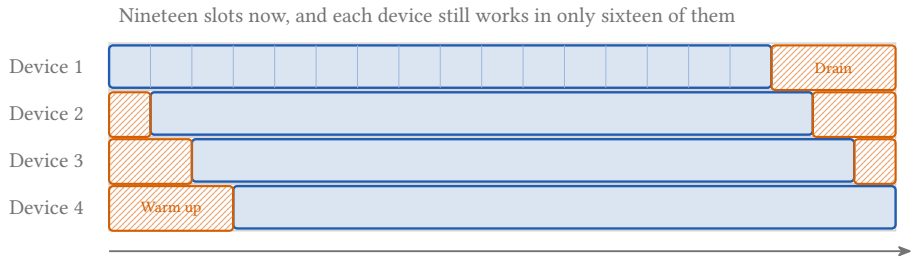


Four stages, one micro-batch



Three slots idle out of four: **75.0 percent** of the machine

Four stages, sixteen micro-batches

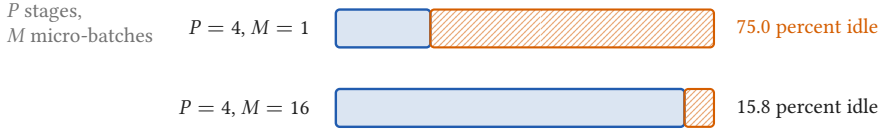


Three slots idle out of nineteen: **15.8 percent**

Each block is one micro-batch passing through one stage. The idle slots per device did not change: the run got longer.

The bubble fraction

$$\text{Idle share} = \frac{\overbrace{P-1}^{\text{slots a device waits}}}{\underbrace{M+P-1}_{\text{slots in the run}}}$$

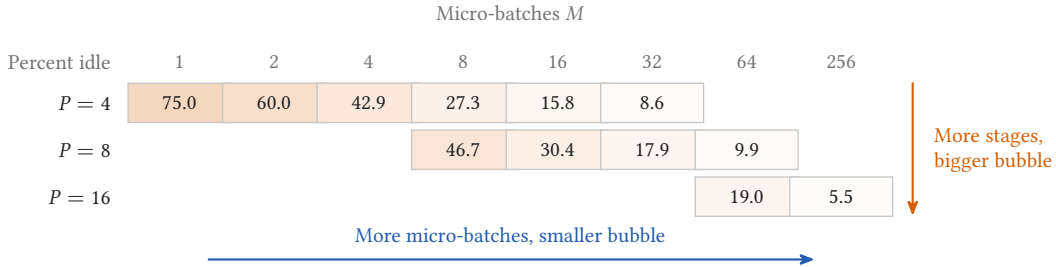


A second convention divides by ideal compute time instead, and reports these same two pipelines as 300.0 and 18.8 percent. Never mix the two.

GPipe (Huang et al. 2019). Assumes balanced stages, free hand-offs, equal micro-batches and a flush per batch.

Cut the depth, and the bubble

More stages makes it worse



- And M is capped: the micro-batches have to add up to the batch size you wanted
- One forward one backward does not shrink the bubble. It lowers peak activation memory from M micro-batches to P, which lets you raise M, and that is what shrinks it.

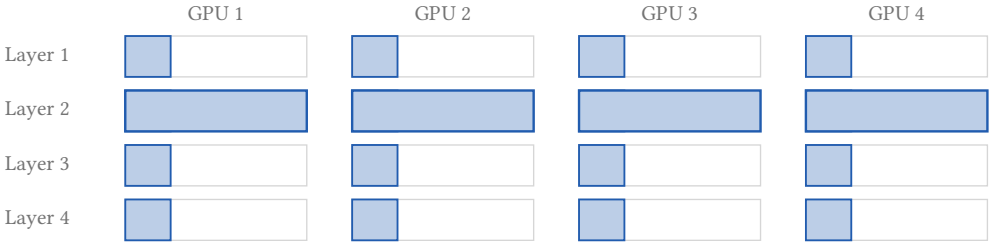
Section 4

Cut the state

4

Cut the state

Nobody stores what they are not using

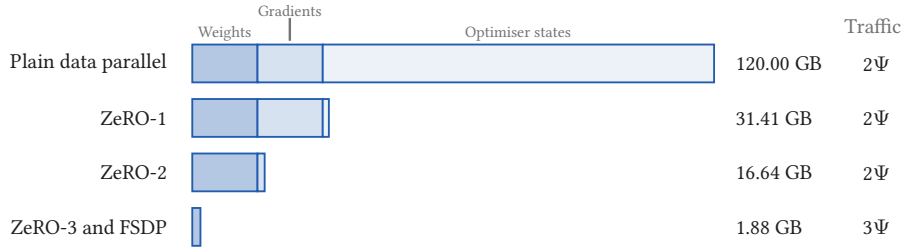


Layer 2 gathered while layer 1 was still computing

- Every device does the same arithmetic on the same batch as before
- Only what is resident changed, and the backward pass ends in a reduce-scatter
- No device holds the whole model now, so part 1's gradient all-gather goes away

Cut the state

Three stages, one bar

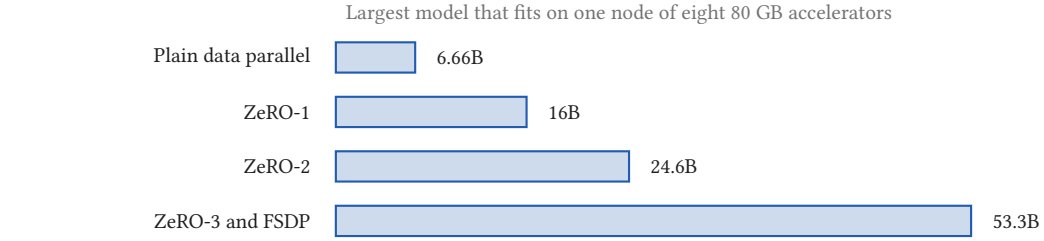


The headline 4x and 8x for the first two stages are limits as the worker count grows.
At 64 workers the realised factors are 3.82x and 7.21x: what is not sharded does not go away.

$\Psi = 7.5 \times 10^9$ parameters, 64 data parallel workers, mixed precision Adam. Traffic in elements, not bytes.

Cut the state

The same eight GPUs, eight times the model



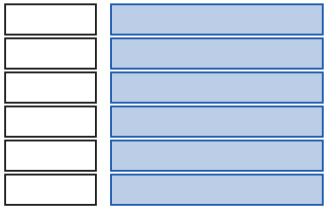
No change to the model, the data or the hardware. One configuration flag.

CS336 Lecture 7, pure bf16 with fp32 master weights, so 12 bytes per parameter here rather than 16.

ZeRO-3 also adds two collectives per layer, which is why the rule of thumb keeps the data parallel degree under about 512.

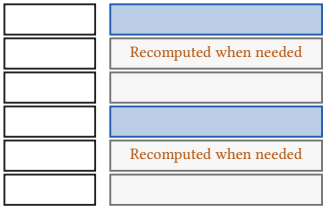
No stage shards the activations

Store every activation



Six layers, six stored

Keep two, recompute the rest



Six layers, two stored

- Activations differ across workers by construction, so no sharding stage touches them
- Full recomputation buys that memory back with a whole extra forward pass: $6N$ per token becomes about $8N$

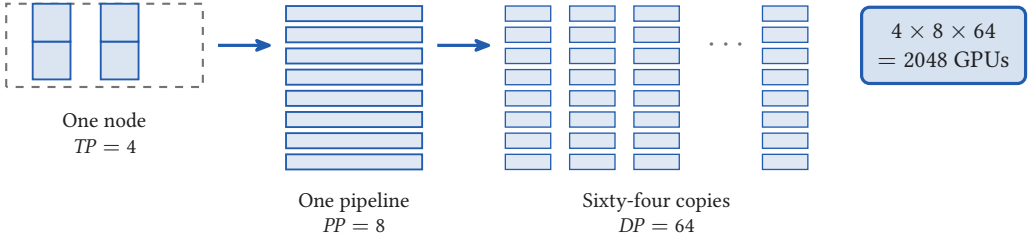
Section 5

All three at once

5

All three at once

Four times eight times sixty-four



Tensor parallel inside the node, pipeline parallel across nodes, data parallel across the copies, and a fourth cut for context on top of those three.

All three at once

A fixed budget, redistributed

Context	*TP*	*PP*	*DP*	*CP*	Product	Tokens/s/GPU
8192	4	8	64	1	2048	780
16384	4	8	32	2	2048	710
32768	4	8	16	4	2048	480
65536	4	8	8	8	2048	160

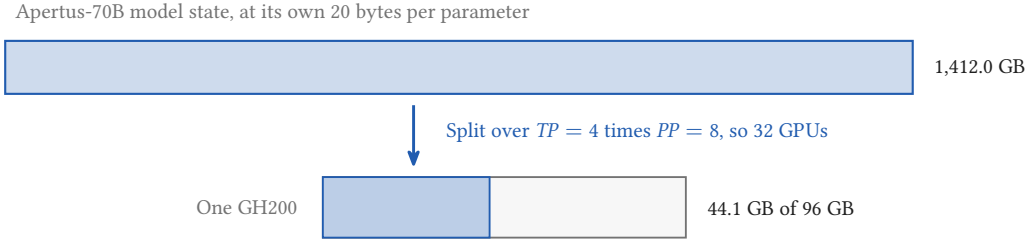
Context parallelism eats data parallelism, row by row, and throughput falls 4.9x as it does

The same 2048 GPUs on every row

Apertus Technical Report Table 5. These are the long context extension phases, not the bulk of pretraining.

All three at once

Back to the ledger



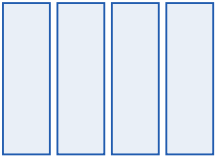
And the rest of that card is where the activations, the context and the buffers live.

All three at once

Next: the exercise at 09:00

Cut the matrix

Cut the depth



Cut the state

- The ledger says whether a configuration fits at all
- The bubble fraction says what a pipeline costs you
- The overlap condition says where a cut is allowed to go
- You make those three choices yourselves next door, and I am staffing it